

Solar Cooker Ray Tracer

Development log — turning a research tool into a shippable application

Branch `feat/interactive-app` · 2026-09-09 · Waves 0-2 of 7 complete

1. What this project was, and what it needed to become

`solar-cooker-rt` was a physics-correct Monte Carlo ray tracer with a thin viewer bolted on. It had done real research — panel-angle sweeps, box-receiver flux maps, a finalised two-level CAD design — but it was not an application. Scenes were authored by **Python scripts writing JSON**. The GUI could browse those files, nudge two sliders per surface, and trace. You could not import a model, delete anything, or save what you edited. Every one of the 94 committed scenes had the sun frozen at the zenith.

The goal: make it usable by a non-technical cooker builder without breaking the JSON/CLI/batch path the research workflow depends on.

Confirmed scope

- **Audience:** mixed, leading with non-technical builders; the expert path stays fully working.
- **Sun:** keep the existing Pillbox/Buie sunshape and manual DNI. Add the ability to move the sun off zenith. No latitude/longitude, no atmospheric model.
- **Results:** optical only — watts, W/m^2 , concentration ratio, flux maps.
- **AI helper:** Claude API with the user's own key. Last priority.
- **Deferred:** thermal model, geographic sun, weather files, day-integrated energy.

Four findings that shaped the plan

Finding	Detail
Scale does <i>not</i> break the <code>t</code> -parameter	<code>Transform::ray_to_local</code> does not normalise the local direction, and <code>t</code> is invariant under any invertible affine map. My initial assumption was wrong; "fixing" it by rescaling <code>t</code> would have <i>introduced</i> the bug.
Polyscope's gizmo is unreachable	<code>TransformationGizmo</code> is a <code>protected</code> member with no public enable method, and has no scale handles. <code>ImGizmo</code> is required instead.
The existing round-trip test was vacuous	It round-tripped <code>nlohmann</code> against itself and never called the loader. Zero real coverage.
Two live bugs blocked runtime mutation	<code>BVH::build</code> retained stale pointers on an empty rebuild; <code>TriangleMesh</code> used an absolute epsilon against a scale-dependent quantity.

2. The plan

Seven sequential waves. Within a wave, file ownership is strictly disjoint — every affected file has exactly one owner.

Wave	Content	Status
0	Foundation & safety net	Complete
1	Sun & aperture physics Scale & surface geometry	Complete
2	SceneDocument & JSON writer — the pivot everything else hangs off	Complete
3	SceneEditor & mutation Viewer decomposition + outliner	Pending
4	ImGuizmo & transform UI model import & units	Pending
5	Async trace, save UI, polish, packaging	Pending
6	AI helper	Pending

3. Wave 0 — Foundation and safety net

No features. Everything downstream depends on being able to tell a fix from a regression.

Latent bugs fixed

- **BVH::build retained dangling pointers.** The early return on an empty surface list sat *above* the `clear()` calls, so a rebuild with no surfaces kept stale `Surface*` that dangle once the old surfaces are freed. Harmless until objects can be deleted at runtime — which is exactly what Wave 3 adds.
- **Two reserve-invariant hazards.** `BVH::build_recursive` and `TriangleMesh::build_tri` hold a reference to a vector element across recursive `emplace_back` calls. Only defined because of an up-front `reserve`. Documented and asserted.
- **Three unguarded null dereferences** in `Viewer::draw_gui`. My original instruction here was *wrong* — I said to guard with an early return, copying a sibling function. An agent caught that `draw_gui` owns the `ImGui::Begin / End` pair, so returning early would unbalance the frame and corrupt rendering. The guards had to be per-block.

The safety net

A golden-flux baseline: four representative scenes traced at a pinned configuration (`rng_seed=42` , `num_threads=1` , 20 000 rays) with the results recorded as constants. Single-threaded matters — it makes the tracer's slot-seeding reduce to exactly `rng_seed` , so the values reproduce across machines.

Scene	Recorded total power
<code>parabolic_dish.json</code>	663.88135955660175 W
<code>box_cooker.json</code>	191.79550384668289 W
<code>fresnel_lens_cooker.json</code>	38.867692665554998 W
<code>stl_rectangular_3floors_606570deg_box.json</code>	874.83793179836209 W

These are *measured*, *not predicted*. Waves 1 and 2 rewrote the aperture power formula, added scaling to every surface type, and replaced the entire scene loader. If any of these four numbers had moved, the optical model changed and that needed explaining rather than re-baselining. They never moved.

Alongside them, a runtime filesystem sweep asserting all **94 committed scenes** load without throwing — replacing a hardcoded list of four filenames.

4. Wave 1 — Sun off zenith, and object scaling

Two workstreams in parallel on disjoint files.

The aperture cosine bug

Ray power was computed as `dni × aperture_area / N`. DNI is defined per unit area *normal to the beam*, so this is only correct when the aperture faces the sun. The moment anyone tilted the sun, total power would be overstated by `1/cos θ` — silently. Every existing scene has the sun at zenith and the aperture normal at +Z, so the cosine is exactly 1.0 and the goldens were unaffected bit-for-bit.

A FLAW IN MY OWN TEST SPECIFICATION

I specified an end-to-end check with the aperture auto-fitted to the sun. But `auto_fit` makes the cosine equal 1 *by construction* — the test could not distinguish a correct cosine factor from a missing one. It would have passed either way.

The agent caught this, added a second case with a **fixed** horizontal aperture struck obliquely, then *falsified* it: it neutered the cosine in the tracer and confirmed the test fails — power stuck at 999 W across all elevations, wrong by +286% at 15°. Then restored the file and diffed against a backup to confirm byte-identity.

Sun elevation	Expected	Measured	Error
90°	1000.000 W	999.224 W	-0.08%
60°	866.025 W	865.353 W	-0.08%
30°	500.000 W	499.612 W	-0.08%
15°	258.819 W	258.618 W	-0.08%

The tolerance was derived from binomial counting statistics rather than guessed: 1% is roughly 11σ — loose enough never to flake, far tighter than the error a wrong cosine produces.

Sun angles and the pole

Azimuth/elevation added with +Z as zenith and azimuth as a compass bearing from +Y (North) toward +X (East) — matching the existing box-receiver wall naming rather than inventing a convention. `cos(90°)` evaluates to $\sim 6.1 \times 10^{-17}$ in double, not zero, so the pole is snapped below a 10^{-12} threshold to keep `direction_from_angles({any, 90°})` *exactly* `(0,0,-1)` — the invariant the goldens rest on.

Scaling

The highest-value change was not the scale factories but **nine relativized degeneracy epsilons**. The worst was the triangle-intersection guard: the tested quantity scales with triangle size *and* with local ray-direction length, so an absolute 10^{-6} threshold silently dropped 1 mm triangles at 100× scale. No crash — just missing geometry and quietly wrong flux.

THE TEST WAS PROVEN TO HAVE TEETH

The agent reverted the guard to its old absolute form and demonstrated the failure: the entire mesh vanishes at scale 1000. Then restored it. A passing test that would pass anyway is worth nothing.

Eight per-class `world_bounds()` implementations were also replaced by one non-virtual base implementation over a new `local_bounds()`, verified bit-identical for all eight classes under a rotated, translated, non-uniformly scaled transform.

Audit A1 — independent review

A fresh agent read the combined diff cold, with no access to either implementer's reasoning. It found a genuine **high-severity** bug:

DECOMPOSE_TRS RETURNED WRONG ANGLES AT PITCH = -90°

A dropped sign factor meant the gimbal-lock branch recovered `-(a-c)` instead of `(a-c)` — while still reporting success. Reproduced with a probe: recomposition error 2.34 versus 1.18×10^{-16} for the correct pole.

The existing test only exercised non-pole angles, so it passed vacuously. Real-world trigger: any panel rotated to face along $-X$, which this project's radial panel arrays produce routinely. **Neither implementer would have found it** — the same agent wrote both the code and the test that failed to catch it.

The audit also confirmed what was *sound*, which matters as much: the cosine is exactly 1.0 bitwise (not merely within tolerance); all eight bounds refactors are byte-identical; all nine epsilons are dimensionally correct; `Aperture::covers` is exact, brute-forced over 200 000 random configurations with zero false results. And it corrected a false claim in the code's own comment — a shared helper described as "byte-identical" actually re-normalises an already-unit vector, differing in the last bit for $\sim 15\%$ of directions.

5. Wave 2 — SceneDocument, the pivot

This wave split the load-only 468-line `SceneLoader.cpp` into a document model, a parser, a writer and a scene builder. Everything downstream — the object editor, save, undo, AI scene validation — depends on it.

Why a document model rather than walking the live scene

Three facts make round-tripping from a built `Scene` lossy: box receivers expand at load into 6 or 11 named faces plus a synthesised absorber; `TriangleMesh` discards its source path and unit scale; and `scene.name` is read by nothing, so a walker would drop it on every save.

What was built

- **The frozen contract** — a type model over all seven surface types, plane and box receivers with the optional battery, materials, sun, aperture, trace config. Material parameters are held as *raw JSON* so open-ended keys survive losslessly, including keys nobody has invented yet.
- **The parser** — every loader default, requirement and alias ported exactly. A strict mode rejects unknown keys at every level, which is what later makes AI-generated scenes safe to validate.
- **The writer** — the project's first C++ scene writer. Always emits `sun.direction`, because the loader throws without it. Emits battery keys only when explicitly authored, so derived defaults are not baked in as though the user had set them.
- **Path handling** — mesh paths resolve against the scene file's own directory, so "Save As" elsewhere would silently break every reference. Rebasing handles the cross-volume case, which on Windows is a realistic user action.
- **Stable identity** — 64-bit IDs on surfaces, removal, lookup, and a BVH dirty flag, since the acceleration structure holds raw pointers that must never be traversed stale.

A BLOCKING GAP CAUGHT BEFORE IT SHIPPED

A sub-agent decided `sun.direction` could be dropped in favour of azimuth/elevation, calling it "an intentional format change". The lead rejected that and verified why: the loader *throws* when the key is absent, and the angle round-trip is bitwise-exact only at the pole. Every scene written without a direction would have been unloadable, and off-zenith scenes would round-trip with perturbed ray sequences — the exact case Wave 1 exists to enable.

A lost-edit defect found across an ownership seam

AUTHORED SUN AZIMUTH SILENTLY DISCARDED

The parser re-derived sun angles from `direction` without passing a fallback azimuth. Azimuth is mathematically degenerate at the pole, so a zenith sun returned the default 180° and any authored `azimuth_deg` was thrown away on save and reload.

Found by the *writer* agent verifying across the seam into the *parser's* file — territory neither it nor its sub-agents owned. Fixed and pinned with a regression test.

Verification

The writer was verified out-of-band over **144 real scenes**: every written document was accepted by the real loader, round-tripping is a fixed point, and all four golden fluxes traced from *rewritten* files came back `delta = 0.000e+00` — bit-identical.

6. Orchestration: what worked and what failed

The work ran as a two-layer agent hierarchy: a commander doing recon, composition and verification; Opus leads owning workstreams; Sonnet sub-agents doing bounded implementation.

What worked

- **Sequential waves with disjoint file ownership.** Two workstreams edited the codebase simultaneously for an entire wave with zero conflicts. Both agents even observed each other's transient build breakage, correctly identified it as the other's in-flight work, and took no action.
- **Independent audits.** Every significant defect in this project was found by an agent that did not write the code. Implementers verify their own assumptions; they cannot verify the ones they never questioned.
- **Golden baselines as a gate.** Four numbers that must not move turned "did I break the physics?" from a judgement call into a test.
- **Refusing to self-certify.** Every completion claim had to cite build output and named tests.

What failed

LAYER-1 LEADS REPORTED BEFORE THEIR SUB-AGENTS FINISHED

The agent tool spawns sub-agents in the background by default, so a lead has nothing to block on and returns immediately — reporting, honestly enough, "my sub-agents are running, I'll verify when they report." But its turn is over.

I misread this as the children being dead and dispatched replacements. **That was wrong.** The children were merely slower than their parents, and delivered good work afterwards — which made my replacements duplicate owners of the same files. I stopped both; one was mid-edit when killed, so the file had to be verified compiling, linking and passing all tests before it could be trusted.

The real defect was narrower than I first said: the leads' *reporting* was broken, not their delegation. The fix is to make leads block on their children, not to abandon the two-layer pattern.

TRUST THE DIFF, NEVER THE REPORT

One lead reported its sub-agents were "actively writing" a file that was still a one-line stub on disk. Had that been taken at face value, the wave would have "completed" with core files never written, surfacing later as unattributable link errors.

Cross-referencing `git diff --stat` against every report is the only thing that catches this. It caught it three times.

A stale escalation, checked rather than acted on

An agent reported that scale and matrix keys were write-only and would silently corrupt geometry — a serious claim. It had read the scene loader *before* the refactor landed. Rather than take either side on faith, the claim was settled with a test: a unit plane scaled by (3,5) loads spanning $x \in [-3, 3]$, $y \in [-5, 5]$ through the real load path. The finding was stale; the test is permanent.

7. Defects found and fixed

Defect	Severity	Found by	Consequence had it shipped
Aperture cosine missing from ray power	High	Plan review	Power overstated by $1/\cos \theta$ for any off-zenith sun, silently
<code>decompose_trs</code> wrong at pitch -90°	High	Independent audit	Any panel facing $-X$ decomposes to a different rotation, reported as success
Authored sun azimuth lost on save	High	Cross-seam verification	User edits silently discarded on reload
Absolute triangle epsilon	Medium	Plan review	Scaled meshes silently drop triangles — missing geometry, wrong flux
<code>BVH::build</code> stale pointers	Medium	Plan review	Dangling pointer traversal once object deletion exists
Mesh assets never packaged	Medium	Packaging check	12 of 20 shipped scenes fail to load in a distributed build
<code>sun.direction</code> nearly dropped from the writer	High	Foundation lead	Every written scene unloadable
Three unguarded null dereferences	Low	Plan review	Crash on an empty scene

A packaging bug worth calling out separately

Twelve of the twenty example scenes reference STL meshes by relative path, but the packaging script only ever copied the scene JSONs. Every STL design — the project's actual finalised work — failed to load in a distributed build. This was present in the previously shipped v1.0.0 archive. Fixed, and verified by tracing an STL scene from inside the packaged folder.

8. Current state

Commit	Description
f7b858b	Research artifacts (35 files)
b880d9b	App track + agent orchestration convention recorded in CLAUDE.md
c0b7fda	Packaging: mesh assets — fixes 12 broken scenes in shipped builds
f15b2a1	Wave 0 — bug fixes, 94-scene corpus, golden baseline
ce0f262	Wave 1 stub registration
b487105	Wave 1 — sun off zenith, aperture cosine, scaling, gimbal fix
9136bbd	Wave 0/1 progress and open audit findings
8e2a0ba	Project landing page
1f5384b	Wave 2 stub registration
e0bf80b	Wave 2 — SceneDocument, JSON writer, stable object IDs
8ed4410	Azimuth-loss fix and first tests covering Wave 2 code

Test suite: 94 cases, 24 248 assertions, all passing. Four golden flux values unchanged across every wave. 94-scene corpus loading clean.

Deliverable: a statically linked, self-contained Windows executable requiring no installation and no runtime dependencies. Double-clicking it auto-discovers the bundled example scenes.

9. Open items

Wave 2 remainder

- The systematic round-trip suite (structural, physics, box-receiver face survival, mesh-path preservation) across all 94 scenes. The evidence exists but lived in agents' scratch harnesses rather than the repository.
- `save_scene` cannot rebase mesh paths — its signature has no room for the original directory. A sibling function can. The Save-As UI must call that one.
- A legacy validation check on receiver top-mode was dropped rather than modify the frozen contract. No scene exercises it, but it is a real capability lost.

Carried from Audit A1

- A below-horizon sun is accepted silently — a fixed aperture yields 0 W with no diagnostic, and auto-fit will cheerfully light the cooker from underground.
- Parameter setters change bounds with no BVH-invalidation hook.
- Two surfaces return an inverted bounding box for negative focal length.
- Downstream Python efficiency formulas assume the cosine is 1, and become wrong by $1/\cos$ once it is reachable from JSON.

- A stale comment claiming byte-identical arithmetic that is not.

10. Lessons

1. **Record a baseline before touching physics.** Four measured numbers converted every subsequent "did this break anything?" from judgement into a test — across a rewritten power formula, scaling on every surface type, and a wholesale loader replacement.
2. **Have someone else check the work.** Every high-severity defect here was found by an agent that had not written the code. The implementer verifies the assumptions they thought to question.
3. **Make tests fail on purpose.** Three separate times a test was proven to have teeth by deliberately reverting the fix and confirming the failure. Two of those tests would otherwise have passed vacuously.
4. **Verify claims in both directions.** One escalated "serious bug" was stale information; one dismissed detail was a genuine data-loss defect. Neither could be resolved by reading the report.
5. **The artefact is the truth.** Reports describe intent. `git diff` describes what happened. They differed three times in this project.